

دانشگاه صنعتی شریف

دانشکده مهندسی کامپیوتر

گزارش آزمایش ۳

توصیف جریان داده

Digital Systems Design Laboratory

اعضای گروه

پوریا عرفانزاده ۴۰۱۱۱۰۹۱۸
فاطیما تیمارچی ۴۰۲۱۰۵۸۰۲
آیه صابری ۴۰۲۱۰۶۱۴۵

نام درس

آزمایشگاه طراحی سیستم‌های دیجیتال

استاد درس

دکتر علیرضا اجلالی

تابستان ۱۴۰۵

۱.۳ مقدمه و اهداف

در این آزمایش، توصیف جریان داده در زبان توصیف سخت‌افزار Verilog با دستور پیوستهٔ assign بررسی می‌شود. بخش نخست شامل طراحی سلسله‌مراتبی مقایسه‌کنندهٔ آبشاری چهاربیتی و بخش دوم شامل طراحی یک مقایسه‌کنندهٔ پی‌درپی دارای ورودی بازنشانی است.

روند انجام آزمایش

برای هر بخش، ابتدا پروژه‌ای جداگانه ساخته و پرونده‌های توصیف سخت‌افزار به آن افزوده شدند. سپس پیمانهٔ سطح بالا تعیین، خانوادهٔ قطعه انتخاب و طرح همگردانی و سنتز شد. در بخش آبشاری، پیمانهٔ یک‌بیتی پیش از پیمانهٔ چهاربیتی ساخته شد؛ در بخش پی‌درپی، همهٔ منطق در یک پیمانه قرار گرفت. پس از ساخت و اجرای میزهای آزمون در ModelSim، خروجی‌ها با نتیجهٔ مرجع مقایسه شدند و در پایان، نمای سطح انتقال ثبات‌ها و گزارش مصرف منابع هر دو طرح بررسی شد.

۲.۳ طراحی و کد توصیف سخت‌افزار

الف) مقایسه‌کنندهٔ آبشاری چهاربیتی

واحد پایه، یک مقایسه‌کنندهٔ یک‌بیتی آبشاری با سه ورودی وضعیّت از مرحلهٔ پرارزش‌تر است. اگر مراحل قبلی برابری را گزارش کرده باشند، بیت جاری می‌تواند نتیجه را به بزرگ‌تر یا کوچک‌تر تغییر دهد؛ در غیر این صورت، نتیجهٔ قطعی مرحلهٔ قبلی بدون تغییر عبور می‌کند. روابط داده‌ای این مرحله عبارت‌اند از:

$$G_{\text{next}} = G_{\text{prev}} \vee (E_{\text{prev}} \wedge A \wedge \overline{B}),$$
$$E_{\text{next}} = E_{\text{prev}} \wedge \overline{A \oplus B},$$
$$L_{\text{next}} = L_{\text{prev}} \vee (E_{\text{prev}} \wedge \overline{A} \wedge B).$$

هر سه رابطه تنها با دستورهای پیوستهٔ assign توصیف شده‌اند.

```
module compare_stage (
    input  wire first_bit,
    input  wire second_bit,

    input  wire greater_from_previous,
    input  wire equal_from_previous,
    input  wire less_from_previous,

    output wire greater_to_next,
    output wire equal_to_next,
    output wire less_to_next
);

    assign greater_to_next =
        greater_from_previous |
        (equal_from_previous & first_bit & ~second_bit);

    assign equal_to_next =
        equal_from_previous & ~(first_bit ^ second_bit);

    assign less_to_next =
        less_from_previous |
        (equal_from_previous & ~first_bit & second_bit);

endmodule
```

شکل ۱: کد جریان داده‌ی مرحله‌ی یک‌بیتی مقایسه‌کننده

برای ساخت مقایسه‌کنندهٔ چهاربیتی، چهار نمونه از این مرحله از بیت ۳ تا بیت ۰ به‌صورت آبشاری متّصل شدند. وضعیّت اوّلیّهٔ مرحلهٔ پرارزش $G = 0, E = 1, L = 0$ است و خروجی هر مرحله به ورودی وضعیّت مرحلهٔ بعد می‌رود. در نتیجه، نخستین اختلاف از سمت بیت پرارزش نتیجهٔ نهایی را تعیین می‌کند.

```
module four_bit_comparator (
    input  wire [3:0] first_number,
    input  wire [3:0] second_number,

    output wire first_is_greater,
    output wire numbers_are_equal,
    output wire first_is_less
);

    wire greater_after_bit3;
    wire equal_after_bit3;
    wire less_after_bit3;


    wire greater_after_bit2;
    wire equal_after_bit2;
    wire less_after_bit2;


    wire greater_after_bit1;
    wire equal_after_bit1;
    wire less_after_bit1;



    compare_stage stage_bit3 (
        .first_bit            (first_number[3]),
        .second_bit           (second_number[3]),

        .greater_from_previous(1'b0),
        .equal_from_previous  (1'b1),
        .less_from_previous   (1'b0),
```

شکل ۲: تعریف ورودی‌ها، خروجی‌ها و مرحله‌ی بیت ۳

```
compare_stage stage_bit3 (
    .first_bit              (first_number[3]),
    .second_bit             (second_number[3]),

    .greater_from_previous(1'b0),
    .equal_from_previous   (1'b1),
    .less_from_previous    (1'b0),

    .greater_to_next       (greater_after_bit3),
    .equal_to_next         (equal_after_bit3),
    .less_to_next          (less_after_bit3)
);

compare_stage stage_bit2 (
    .first_bit              (first_number[2]),
    .second_bit             (second_number[2]),

    .greater_from_previous(greater_after_bit3),
    .equal_from_previous   (equal_after_bit3),
    .less_from_previous    (less_after_bit3),

    .greater_to_next       (greater_after_bit2),
    .equal_to_next         (equal_after_bit2),
    .less_to_next          (less_after_bit2)
);
```

شکل ۳: اتّصال مرحلهٔ بیت ۳ به مرحلهٔ بیت ۲

```
    compare_stage stage_bit1 (
        .first_bit              (first_number[1]),
        .second_bit             (second_number[1]),

        .greater_from_previous(greater_after_bit2),
        .equal_from_previous   (equal_after_bit2),
        .less_from_previous    (less_after_bit2),

        .greater_to_next       (greater_after_bit1),
        .equal_to_next         (equal_after_bit1),
        .less_to_next          (less_after_bit1)
    );

    compare_stage stage_bit0 (
        .first_bit              (first_number[0]),
        .second_bit             (second_number[0]),

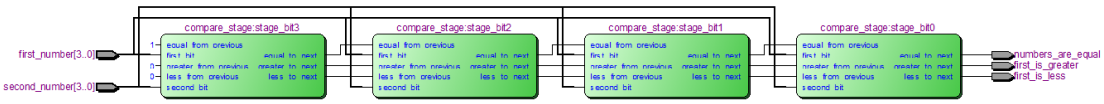
        .greater_from_previous(greater_after_bit1),
        .equal_from_previous   (equal_after_bit1),
        .less_from_previous    (less_after_bit1),

        .greater_to_next       (first_is_greater),
        .equal_to_next         (numbers_are_equal),
        .less_to_next          (first_is_less)
    );

endmodule
```

شکل ۴: اتّصال مراحل بیت ۱ و صفر و تولید خروجی نهایی

نمای سطح انتقال ثبات‌ها دقیقاً چهار نمونه از پیمانهٔ compare_stage را به‌صورت متوالی نشان می‌دهد و بنابراین ساختار سلسله‌مراتبی موردنظر دستورکار حفظ شده است.



شکل ۵: نمای سطح انتقال ثبات‌های مقایسه‌کنندهٔ آبشاری چهاربیتی

ب) مقایسه‌کنندهٔ پی‌درپی

مقایسه‌کنندهٔ پی‌درپی پس از فعّال‌شدن ورودی بازنشانی reset در حالت برابر، یعنی $G = 0, E = 1, L = 0$، قرار می‌گیرد. بیت‌های دو عدد از پرارزش‌ترین بیت به کم‌ارزش‌ترین بیت وارد می‌شوند. تا وقتی $E = 1$ است، هر بیت جدید می‌تواند رابطه را تعیین کند؛ پس از مشاهدهٔ نخستین اختلاف، وضعیّت بزرگ‌تر یا کوچک‌تر حفظ می‌شود. روابط حالت بعد همان روابط مرحلهٔ آبشاری‌اند، امّا نتیجه باید بین پالس‌های ساعت نگهداری شود.

برای رعایت محدودیّت استفادهٔ انحصاری از assign، نگهداری وضعیّت با شش شبکهٔ بازخوردی در دو مرحلهٔ اصلی و تابع انجام شده است. در یک سطح ساعت، حالت بعد در سیگنال‌های مرحلهٔ اصلی با پیشوند master_ ذخیره می‌شود و در سطح دیگر به خروجی منتقل می‌گردد. بازنشانی نیز به‌صورت غیرهم‌زمان مقادیر اوّلیّه را تحمیل می‌کند. مقایسه‌کنندهٔ پی‌درپی در یک پیمانه و بدون نمونه‌سازی هیچ پیمانهٔ دیگری پیاده‌سازی شده است؛ بنابراین تمام منطق آن، مطابق محدودیّت دستورکار، بدون توصیف رفتاری و فقط با توصیف جریان داده ساخته شده است.

```
module serial_bit_comparator (
    input  wire reset,
    input  wire clock,

    input  wire serial_first_bit,
    input  wire serial_second_bit,

    output wire first_is_greater,
    output wire bits_are_equal,
    output wire first_is_less
);

    wire next_greater;
    wire next_equal;
    wire next_less;

    wire master_greater;
    wire master_equal;
    wire master_less;

    assign next_greater =
        first_is_greater |
        (bits_are_equal & serial_first_bit & ~serial_second_bit);

    assign next_equal =
        bits_are_equal &
        ~(serial_first_bit ^ serial_second_bit);

    assign next_less =
        first_is_less |
        (bits_are_equal & ~serial_first_bit & serial_second_bit);

    assign master_greater =
```

شکل ۶: تعریف رابط و روابط حالت بعد مقایسه‌کنندهٔ پی‌درپی

```
    assign next_less =
        first_is_less |
        (bits_are_equal & ~serial_first_bit & serial_second_bit);

    assign master_greater =
        reset ? 1'b0 :
        (clock ? master_greater : next_greater);

    assign first_is_greater =
        reset ? 1'b0 :
        (clock ? master_greater : first_is_greater);

    assign master_equal =
        reset ? 1'b1 :
        (clock ? master_equal : next_equal);

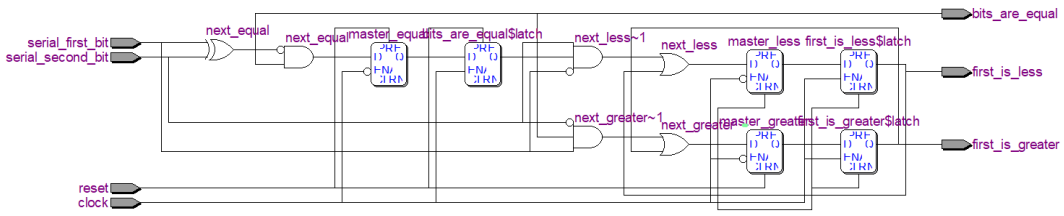
    assign bits_are_equal =
        reset ? 1'b1 :
        (clock ? master_equal : bits_are_equal);

    assign master_less =
        reset ? 1'b0 :
        (clock ? master_less : next_less);

    assign first_is_less =
        reset ? 1'b0 :
        (clock ? master_less : first_is_less);

endmodule
```

شکل ۷: پیاده‌سازی نگهداری حالت و بازنشانی تنها با assign



شکل ۸: نمای سطح انتقال ثبات‌ها و عناصر حافظهٔ استنتاج‌شده برای مقایسه‌کنندهٔ پی‌درپی

۳.۳ میز آزمون و شبیه‌سازی

الف) آزمون مقایسه‌کننده‌ی آبشاری

میز آزمون آبشاری تمام $16 \times 16 = 256$ ترکیب ممکن دو ورودی چهاربیتی را پیمایش می‌کند. برای هر بردار، سه خروجی مدار با عملگرهای مرجع $>$، $==$ و $<$ مقایسه شده و هر مغایرت در شمارندهٔ خطای error_count ثبت می‌شود.

```
`timescale 1ns/1ps

module four_bit_comparator_tb;

    reg  [3:0] test_first_number;
    reg  [3:0] test_second_number;

    wire result_greater;
    wire result_equal;
    wire result_less;

    integer first_value;
    integer second_value;
    integer error_count;

    four_bit_comparator dut (
        .first_number      (test_first_number),
        .second_number     (test_second_number),
        .first_is_greater  (result_greater),
        .numbers_are_equal (result_equal),
        .first_is_less     (result_less)
    );

    initial begin

        test_first_number  = 4'b0000;
        test_second_number = 4'b0000;
        error_count        = 0;

        $display("-------------------------------------------");
        $display("Starting exhaustive comparator simulation");
        $display("-------------------------------------------");
```

شکل ۹: تعریف میز آزمون و نمونه‌سازی مقایسه‌کنندهٔ چهاربیتی

```
for (first_value = 0; first_value < 16; first_value = first_value + 1) begin
    for (second_value = 0; second_value < 16; second_value = second_value + 1) begin

        test_first_number  = first_value;
        test_second_number = second_value;

        #10;

        if (
            (result_greater !== (test_first_number > test_second_number)) ||
            (result_equal   !== (test_first_number == test_second_number)) ||
            (result_less    !== (test_first_number < test_second_number))
        ) begin

            error_count = error_count + 1;

            $display(
                "ERROR: first=%d second=%d | greater=%b equal=%b less=%b",
                test_first_number,
                test_second_number,
                result_greater,
                result_equal,
                result_less
            );

        end

    end
end
```

شکل ۱۰: حلقه‌های آزمون فراگیر و مقایسه با نتیجه‌ی مرجع

```
        if (error_count == 0) begin
            $display("------------------------------------------------");
            $display("ALL 256 TEST CASES PASSED SUCCESSFULLY");
            $display("------------------------------------------------");
        end
        else begin
            $display("------------------------------------------------");
            $display("SIMULATION FAILED: %d errors detected", error_count);
            $display("------------------------------------------------");
        end

        $stop;

    end

endmodule
```

شکل ۱۱: گزارش قبولی یا خطای آزمون فراگیر

اجرای میز آزمون در ModelSim با همگردانی سه پیمانه و افزودن سیگنال‌ها به پنجرهٔ موج انجام شد. خروجی پایانه، قبولی تمام ۲۵۶ حالت و صفر بودن شمارندهٔ خطا را نشان می‌دهد. شکل موج کامل و نمای پالایش‌شده نیز ویژگی تک‌فعّال خروجی‌ها را تأیید می‌کنند؛ در هر لحظه دقیقاً یکی از سه رابطهٔ بزرگ‌تر، برابر یا کوچک‌تر فعّال است.

```
# 3 compiles, 0 failed with no errors.
ModelSim> vsim work.four_bit_comparator_tb
# vsim work.four_bit_comparator_tb
# Loading work.four_bit_comparator_tb
# Loading work.four_bit_comparator
# Loading work.compare_stage
VSIM 2> add wave -r sim:/four_bit_comparator_tb/*
VSIM 3> run -all
# ----------------------------------------
# Starting exhaustive comparator simulation
# ----------------------------------------
# ----------------------------------------
# ALL 256 TEST CASES PASSED SUCCESSFULLY
# ----------------------------------------
# Break in Module four_bit_comparator_tb at C:/Users/ASUS/Downloads/dsd3/ModelSim/Exp_3_simulation/Cascadable_Comparator_Simulation/four_bit_comparator_tb.v line 75

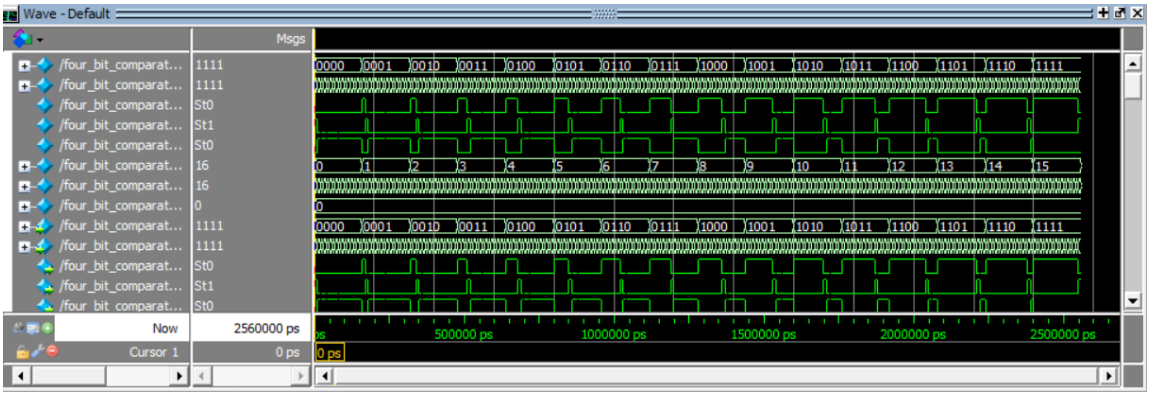
VSIM 4>
```

شکل ۱۲: فرمان‌های اجرای میز آزمون در ModelSim

```
Transcript
# ----------------------------------------
# Break in Module four_bit_comparator_tb at C:/Users/ASUS/Downloads/dsd3/ModelSim/Exp_3_simulation/Cascadable_Comparator_Simulation/four_bit_comparator_tb.v line 75
VSIM 6> view wave
# .main_pane.wave.interior.cs.body.pw.wf
VSIM 7> restart -f
VSIM 8> add wave -r sim:/four_bit_comparator_tb/*
VSIM 9> add wave -r sim:/four_bit_comparator_tb/dut/*
VSIM 10> run -all
# ----------------------------------------
# Starting exhaustive comparator simulation
# ----------------------------------------
# ----------------------------------------
# ALL 256 TEST CASES PASSED SUCCESSFULLY
# ----------------------------------------
# Break in Module four_bit_comparator_tb at C:/Users/ASUS/Downloads/dsd3/ModelSim/Exp_3_simulation/Cascadable_Comparator_Simulation/four_bit_comparator_tb.v line 75

VSIM 11>
```

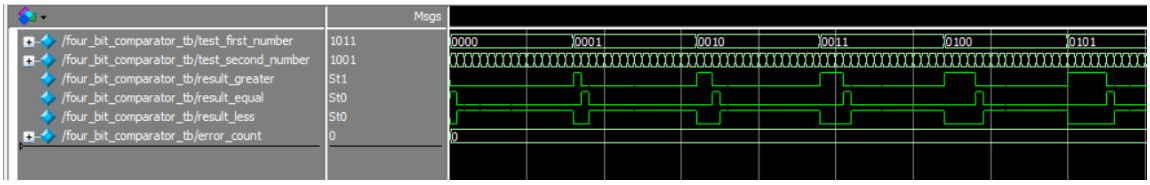
شکل ۱۳: پیام قبولی همه‌ی ۲۵۶ حالت آزمون



شکل ۱۴: شکل موج همه‌ی سیگنال‌های مقایسه‌کننده‌ی آبشاری



شکل ۱۵: شکل موج ورودی‌ها و خروجی‌های اصلی



شکل ۱۶: نمای بزرگ‌شده‌ی تغییر خروجی‌ها برای چند بردار نخست

ب) آزمون مقایسه‌کنندهٔ پی‌درپی

میز آزمون پی‌درپی یک ساعت با نیم‌دورهٔ ۵ نانوثانیه تولید می‌کند. وظیفهٔ reset_comparator درستی حالت اوّلیّه را کنترل می‌کند و وظیفهٔ compare_words بیت‌های دو کلمهٔ چهاربیتی را از بیت ۳ تا صفر، در لبهٔ منفی، اعمال می‌کند. خروجی پس از لبهٔ مثبت با یک الگوی مرجع مقایسه می‌شود؛ الگوی مرجع نیز فقط تا زمان برابر بودن پیشوند، اجازهٔ تغییر نتیجه را می‌دهد.

```
`timescale 1ns/1ps

module serial_bit_comparator_tb;

    reg clock;
    reg reset;

    reg serial_first_bit;
    reg serial_second_bit;

    wire first_is_greater;
    wire bits_are_equal;
    wire first_is_less;

    integer error_count;

    serial_bit_comparator dut (
        .reset             (reset),
        .clock             (clock),
        .serial_first_bit  (serial_first_bit),
        .serial_second_bit (serial_second_bit),
        .first_is_greater  (first_is_greater),
        .bits_are_equal    (bits_are_equal),
        .first_is_less     (first_is_less)
    );


    initial begin
        clock = 1'b0;

        forever begin
            #5 clock = ~clock;
        end
```

شکل ۱۷: تعریف سیگنال‌ها، مدار تحت آزمون و مولّد ساعت

```
end

task reset_comparator;
    begin
        reset             = 1'b1;
        serial_first_bit  = 1'b0;
        serial_second_bit = 1'b0;

        #2;

        if (
            first_is_greater !== 1'b0 ||
            bits_are_equal   !== 1'b1 ||
            first_is_less    !== 1'b0
        ) begin
            error_count = error_count + 1;

            $display(
                "RESET ERROR: greater=%b equal=%b less=%b",
                first_is_greater,
                bits_are_equal,
                first_is_less
            );
        end

        @(negedge clock);
        #1;
        reset = 1'b0;
    end
endtask

task compare_words;
```

شکل ۱۸: وظیفهٔ بازنشانی و کنترل حالت اوّلیّه

```
        input [3:0] first_word;
        input [3:0] second_word;

        integer bit_index;

        reg expected_greater;
        reg expected_equal;
        reg expected_less;

        begin
            expected_greater = 1'b0;
            expected_equal   = 1'b1;
            expected_less    = 1'b0;

            reset_comparator;

            $display(
                "Testing first=%b second=%b",
                first_word,
                second_word
            );

            for (
                bit_index = 3;
                bit_index >= 0;
                bit_index = bit_index - 1
            ) begin

                @(negedge clock);

                serial_first_bit  = first_word[bit_index];
                serial_second_bit = second_word[bit_index];
```

شکل ۱۹: تعریف وظیفهٔ مقایسه و ارسال بیت‌ها از بیت پرارزش

```
            if (expected_equal == 1'b1) begin

                if (first_word[bit_index] > second_word[bit_index]) begin
                    expected_greater = 1'b1;
                    expected_equal   = 1'b0;
                    expected_less    = 1'b0;
                end
                else if (first_word[bit_index] < second_word[bit_index]) begin
                    expected_greater = 1'b0;
                    expected_equal   = 1'b0;
                    expected_less    = 1'b1;
                end

            end

            @(posedge clock);
            #1;

            if (
                first_is_greater !== expected_greater ||
                bits_are_equal   !== expected_equal   ||
                first_is_less    !== expected_less
            ) begin

                error_count = error_count + 1;

                $display(
                    "ERROR at bit %0d: A_bit=%b B_bit=%b | actual=%b%b%b expected=%b%b%b",
                    bit_index,
                    serial_first_bit,
                    serial_second_bit,
                    first_is_greater,
                    bits_are_equal,
```

شکل ۲۰: به‌روزرسانی الگوی مرجع و مقایسهٔ خروجی‌ها

```
                    serial_second_bit,
                    first_is_greater,
                    bits_are_equal,
                    first_is_less,
                    expected_greater,
                    expected_equal,
                    expected_less
                );

            end
            else begin

                $display(
                    "Bit %0d passed: A_bit=%b B_bit=%b result=%b%b%b",
                    bit_index,
                    serial_first_bit,
                    serial_second_bit,
                    first_is_greater,
                    bits_are_equal,
                    first_is_less
                );

            end

        end
    end
endtask

initial begin

    reset             = 1'b1;
    serial_first_bit  = 1'b0;
    serial_second_bit = 1'b0;
```

شکل ۲۱: گزارش نتیجهٔ هر بیت و پایان وظیفه

```
        reset             = 1'b1;
        serial_first_bit  = 1'b0;
        serial_second_bit = 1'b0;
        error_count       = 0;

        $display("========================================");
        $display("Starting serial comparator simulation");
        $display("========================================");

        compare_words(4'b1100, 4'b1100);

        compare_words(4'b1000, 4'b0111);

        compare_words(4'b0100, 4'b0111);

        compare_words(4'b1011, 4'b1001);

        compare_words(4'b0011, 4'b0100);

        compare_words(4'b1011, 4'b1010);

        $display("========================================");

        if (error_count == 0) begin
            $display("ALL SERIAL COMPARATOR TESTS PASSED");
        end
        else begin
            $display(
                "SERIAL SIMULATION FAILED: %0d errors",
                error_count
            );
        end
```

شکل ۲۲: بردارهای آزمون برابر، بزرگ‌تر و کوچک‌تر

```
        $display("========================================");

        if (error_count == 0) begin
            $display("ALL SERIAL COMPARATOR TESTS PASSED");
        end
        else begin
            $display(
                "SERIAL SIMULATION FAILED: %0d errors",
                error_count
            );
        end

        $display("========================================");

        $stop;

    end

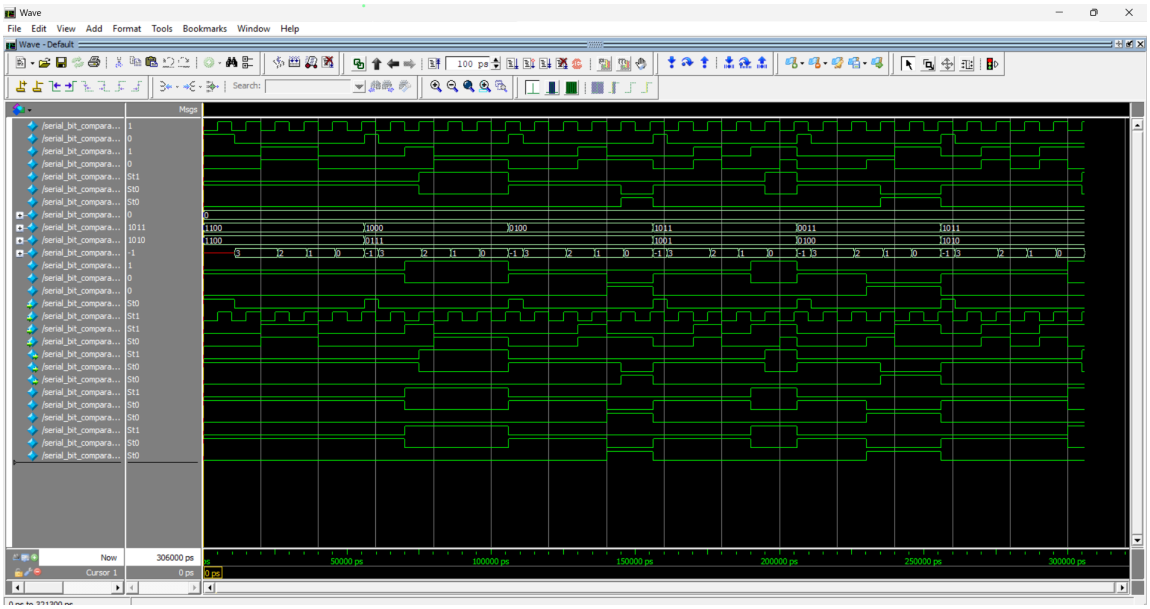
endmodule
```

شکل ۲۳: گزارش نهایی قبولی یا تعداد خطاها

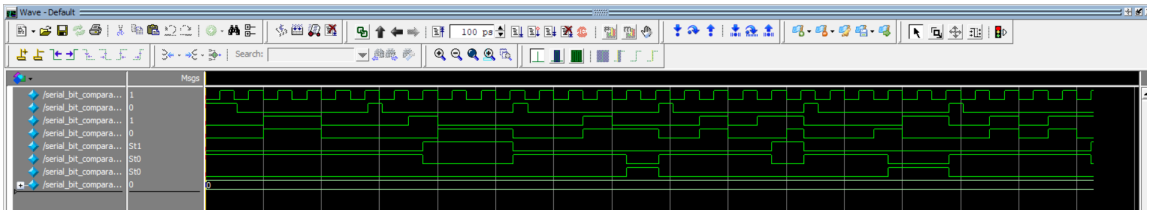
شش زوج آزمون شامل حالت برابر و نمونه‌های بزرگ‌تر و کوچک‌تر اعمال شدند. متن پایانه نتیجهٔ هر بیت را نشان می‌دهد و در پایان، پیام قبولی همهٔ آزمون‌ها چاپ شده است. شکل موج‌ها نیز بازنشانی به 010، دریافت پی‌درپی بیت‌ها و تثبیت نتیجه پس از نخستین اختلاف معنادار را نشان می‌دهند.

```
Transcript
VSIM 3> delete wave *
VSIM 4> run -all
# ============================================
# Starting serial comparator simulation
# ============================================
# Testing first=1100 second=1100
# Bit 3 passed: A_bit=1 B_bit=1 result=010
# Bit 2 passed: A_bit=1 B_bit=1 result=010
# Bit 1 passed: A_bit=0 B_bit=0 result=010
# Bit 0 passed: A_bit=0 B_bit=0 result=010
# Testing first=1000 second=0111
# Bit 3 passed: A_bit=1 B_bit=0 result=100
# Bit 2 passed: A_bit=0 B_bit=1 result=100
# Bit 1 passed: A_bit=0 B_bit=1 result=100
# Bit 0 passed: A_bit=0 B_bit=1 result=100
# Testing first=0100 second=0111
# Bit 3 passed: A_bit=0 B_bit=0 result=010
# Bit 2 passed: A_bit=1 B_bit=1 result=010
# Bit 1 passed: A_bit=0 B_bit=1 result=001
# Bit 0 passed: A_bit=0 B_bit=1 result=001
# Testing first=1011 second=1001
# Bit 3 passed: A_bit=1 B_bit=1 result=010
# Bit 2 passed: A_bit=0 B_bit=0 result=010
# Bit 1 passed: A_bit=1 B_bit=0 result=100
# Bit 0 passed: A_bit=1 B_bit=1 result=100
# Testing first=0011 second=0100
# Bit 3 passed: A_bit=0 B_bit=0 result=010
# Bit 2 passed: A_bit=0 B_bit=1 result=001
# Bit 1 passed: A_bit=1 B_bit=0 result=001
# Bit 0 passed: A_bit=1 B_bit=0 result=001
# Testing first=1011 second=1010
# Bit 3 passed: A_bit=1 B_bit=1 result=010
# Bit 2 passed: A_bit=0 B_bit=0 result=010
# Bit 1 passed: A_bit=1 B_bit=1 result=010
# Bit 0 passed: A_bit=1 B_bit=0 result=100
# ============================================
# ALL SERIAL COMPARATOR TESTS PASSED
```

شکل ۲۴: خروجی پایانه و قبولی تمام آزمون‌های پی‌درپی



شکل ۲۵: شکل موج کامل سیگنال‌های داخلی و میز آزمون



شکل ۲۶: شکل موج سیگنال‌های ضروری مقایسه‌کنندهٔ پی‌درپی

۴.۳ سنتز و پیاده‌سازی نرم‌افزاری

هر دو طرح برای قطعهٔ EP2C5T144C6 از خانوادهٔ Cyclone II سنتز شدند. مقایسه‌کنندهٔ آبشاری ۹ تابع ترکیبی و ۱۱ پایه، و مقایسه‌کنندهٔ پی‌درپی ۹ تابع ترکیبی و ۷ پایه مصرف می‌کند؛ در هر دو مورد، مصرف عناصر منطقی کمتر از یک درصد ظرفیّت تراشه است.

Flow Summary	
Flow Status	Successful - Wed Aug 05 11:27:26 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	top
Top-level Entity Name	four_bit_comparator
Family	Cyclone II
Total logic elements	9 / 4,608 (< 1 %)
Total combinational functions	9 / 4,608 (< 1 %)
Dedicated logic registers	0 / 4,608 (0 %)
Total registers	0
Total pins	11 / 89 (12 %)
Total virtual pins	0
Total memory bits	0 / 119,808 (0 %)
Embedded Multiplier 9-bit elements	0 / 26 (0 %)
Total PLLs	0 / 2 (0 %)
Device	EP2C5T144C6
Timing Models	Final

شکل ۲۷: خلاصه‌ی منابع مقایسه‌کننده‌ی آبشاری

Flow Summary	
Flow Status	Successful - Wed Aug 05 13:14:01 2026
Quartus II 64-Bit Version	13.0.1 Build 232 06/12/2013 SP 1 SJ Web Edition
Revision Name	serial_bit_comparator
Top-level Entity Name	serial_bit_comparator
Family	Cyclone II
Total logic elements	9 / 4,608 (< 1 %)
Total combinational functions	9 / 4,608 (< 1 %)
Dedicated logic registers	0 / 4,608 (0 %)
Total registers	0
Total pins	7 / 89 (8 %)
Total virtual pins	0
Total memory bits	0 / 119,808 (0 %)
Embedded Multiplier 9-bit elements	0 / 26 (0 %)
Total PLLs	0 / 2 (0 %)
Device	EP2C5T144C6
Timing Models	Final

شکل ۲۸: خلاصهٔ منابع مقایسه‌کنندهٔ پی‌درپی

همگردانی هر دو پروژه با صفر خطا تکمیل شد. طرح آبشاری به‌دلیل ترکیبی بودن ساعت ندارد و هشدارهای تحلیل زمان برای آن طبیعی است. برای طرح پی‌درپی قید زمانی مستقلی تعریف نشده است؛ ازاین‌رو، هشدارهای تحلیل زمان و کمینهٔ پهنای پالس باقی مانده‌اند. همچنین در پرونده‌های پیکربندی پروژه تخصیص پایه‌ای برای کلیدها، ساعت و چراغ‌های برد ثبت نشده است. با توجّه به برگزاری مجازی آزمایشگاه، شبیه‌سازی و سنتز نرم‌افزاری مبنای ارزیابی این آزمایش بوده و ادّعایی دربارهٔ اجرای واقعی روی برد مطرح نمی‌شود.

Type	ID	Message
ⓘ	332142	No user constrained base clocks found in the design. Calling "derive_clocks -per
ⓘ	332096	The command derive_clocks did not find any clocks to derive. No clocks were cre
⚠	332068	No clocks defined in design.
ⓘ	332140	No Setup paths to report
ⓘ	332140	No Hold paths to report
ⓘ	332140	No Recovery paths to report
ⓘ	332140	No Removal paths to report
ⓘ	332140	No Minimum Pulse Width paths to report
ⓘ	332001	The selected device family is not supported by the report_metastability command.
ⓘ	332102	Design is not fully constrained for setup requirements
ⓘ	332102	Design is not fully constrained for hold requirements
ⓘ		Quartus II 64-Bit TimeQuest Timing Analyzer was successful. 0 errors, 4 warnings
ⓘ	293000	Quartus II Full Compilation was successful. 0 errors, 12 warnings

شکل ۲۹: همگردانی موفّق مقایسه‌کنندهٔ آبشاری

```
332146 Worst-case hold slack is 0.762
332140 No Recovery paths to report
332140 No Removal paths to report
332146 Worst-case minimum pulse width slack is -1.380
332001 The selected device family is not supported by the report_metastability command.
332102 Design is not fully constrained for setup requirements
332102 Design is not fully constrained for hold requirements
       Quartus II 64-Bit TimeQuest Timing Analyzer was successful. 0 errors, 5 warnings
293000 Quartus II Full Compilation was successful. 0 errors, 13 warnings
```

شکل ۳۰: همگردانی موفّق مقایسه‌کنندهٔ پی‌درپی

۵.۳ نتایج

- مقایسه‌کنندهٔ چهاربیتی با اتّصال سلسله‌مراتبی چهار مرحلهٔ یک‌بیتی و صرفاً با توصیف جریان داده ساخته شد.
- آزمون فراگیر تمام ۲۵۶ زوج ورودی بدون خطا پایان یافت.
- مقایسه‌کنندهٔ پی‌درپی نتیجهٔ مقایسهٔ پیشوند دریافت‌شده را در هر پالس نگه داشت و پس از نخستین اختلاف، وضعیّت نهایی را حفظ کرد.
- حالت بازنشانی $G = 0, E = 1, L = 0$ و هر شش زوج آزمون پی‌درپی با موفّقیّت بررسی شدند.
- هر دو پروژه بدون خطای همگردانی و با مصرف کمتر از یک درصد منابع منطقی سنتز شدند.

نتیجه‌گیری

این آزمایش نشان داد که توصیف جریان داده هم برای مدار ترکیبی سلسله‌مراتبی و هم، با استفاده از بازخورد کنترل‌شده، برای مدار ترتیبی قابل‌استفاده است. زنجیرهٔ مقایسه از بیت پرارزش به کم‌ارزش، منطق اولویت عددی را مستقیماً پیاده‌سازی کرد و نسخهٔ پی‌درپی همان منطق را در طول زمان نگه داشت. میز آزمون فراگیر بخش اوّل، بررسی بیت‌به‌بیت بخش دوم و سنتز موفّق هر دو طرح، درستی عملکرد را مطابق اهداف آزمایش مجازی تأیید کردند.